

One-to-one SkewGRAM: from biology to NLP

Bechir Mady¹, Abderahmane Abderrahmane¹, Mohamedou Yahya Cheikh
Mohamed Vall¹, El Moustapha Mohamed El Moustapha¹, Mohamed Lemine
Ahmed Sidi¹, and Hamed Mohamed Babou²

¹ ¹Unité de Recherche en Systèmes Intelligents Avancés (URSIA), Institut Supérieur
du Numérique (SupNum), Nouakchott, Mauritania

{22014, 22053, 25239, 22013}@supnum.mr, m-lamine.ahmed-sidi@supnum.mr

² ²École Supérieure Polytechnique (ESP), Nouakchott, Mauritania
hamed.mohamed-babou@esp.mr

Abstract. The One-to-one SkewGRAM problem is an NP-hard combinatorial optimization problem arising in bioinformatics. Given a directed acyclic graph D and an undirected graph G defined on the same set of biological entities, the objective is to find a longest path in D whose vertices induce a connected subgraph in G . Exact optimization methods can provide optimal solutions but may become computationally expensive, while recent Graph Neural Network approaches tend to generate very short paths.

We propose **One-to-one SkewGRAM**, a Skip-Gram-style node embedding approach based on directed random walks over D . The learned representations are combined with a guided decoder integrating three complementary signals: an exact dynamic-programming lookahead on D , embedding similarity, and a connectivity bonus based on G . Since these signals guide the search without enforcing the global connectivity constraint, a final exact sliding-window procedure based on Union-Find extracts the longest contiguous sub-path that satisfies the connectivity requirement.

The proposed approach therefore combines representation learning with exact graph algorithms: embeddings guide the exploration, dynamic programming preserves the potential of the directed path, and the final extraction step guarantees the required connectivity. Experimental evaluation on synthetic instances is used to assess its solution quality and computational efficiency against exact and learning-based approaches.

Keywords: Combinatorial Optimization · Bioinformatics · Graph Algorithms · Node Embeddings · Skip-Gram · Dynamic Programming · Connected Components · Union-Find.

1 Introduction

Artificial Intelligence (AI) and machine learning are increasingly used in bioinformatics to analyse complex biological data. Biological systems can naturally be represented as networks, where vertices correspond to biological entities and

edges represent relationships between them. Combining complementary network structures is particularly useful for discovering conserved functional patterns.

For example, cellular metabolism can be represented by a directed graph, where metabolites are connected through directed biochemical reactions describing a causal flow. At the same time, genomic proximity or protein-protein interactions (PPI) [7] can be represented by an undirected graph describing functional associations. Identifying structures that are simultaneously meaningful in these two graphs can therefore reveal important biological patterns.

The **One-to-one SkewGRAM** problem formalizes this task. Given a directed acyclic graph D and an undirected graph G over the same set of vertices, the objective is to find a longest directed path in D whose vertices induce a connected subgraph in G . This problem is NP-hard [1,8].

The difficulty comes from the interaction between two different structural constraints. A valid successor in D does not necessarily preserve connectivity in G , while vertices that are functionally connected in G may not form a directed path in D .

Recent work has investigated deep-learning approaches to this problem [2]. In particular, a dual-GCN architecture with heuristic post-processing and a GCN encoder combined with an LSTM-Attention decoder (DL2) have been studied. Although these approaches demonstrate the potential of learning-based methods, the sequential decoding strategy generates very short paths, with DL2 producing approximately 1.2 nodes on average. In contrast, the exact ILP2 solver [1] obtains optimal solutions but requires approximately 28 seconds per 100-node instance.

These limitations motivate a hybrid approach that combines learned representations with explicit graph algorithms. Our main idea is to reinterpret directed random walks over D as sentences, following the Skip-Gram and DeepWalk/node2vec paradigm [4,5,6]. Nodes play the role of words, while nodes occurring in similar contexts receive related representations.

The learned embeddings alone, however, are not sufficient to solve the complete optimization problem. We therefore use them as a guidance signal inside a problem-specific decoder. The decoder combines embedding similarity with an exact dynamic-programming lookahead on D and a soft connectivity signal from G . A final exact extraction step verifies the connectivity constraint.

The contributions of this paper are:

1. A Skip-Gram-style node embedding model for DAG nodes, using directed random walks, hub sub-sampling, and structural negative sampling.
2. A guided decoder combining exact DAG longest-path lookahead, learned embedding similarity, and a G -connectivity bonus.
3. An exact sliding-window Union-Find procedure that extracts the longest (D, G) -consistent sub-path from each generated candidate.
4. An empirical evaluation showing a substantial quality–time trade-off compared with the exact ILP2 solver and the previously reported GNN approach.

2 Problem Formulation and Mathematical Property

Let

$$V = \{v_1, \dots, v_n\}$$

be a set of biological entities. We consider two graphs defined over V :

- $D = (V, A)$: a directed acyclic graph modeling a causal or sequential relationship;
- $G = (V, E)$: an undirected graph modeling functional associations.

The objective is to find a path

$$P = (p_1, \dots, p_k)$$

maximizing its number of vertices, subject to:

$$(p_i, p_{i+1}) \in A, \quad i = 1, \dots, k-1, \quad (1)$$

and

$$G[\{p_1, \dots, p_k\}] \text{ is connected.} \quad (2)$$

The path is therefore valid in D while its vertex set is connected in G . We call such a path a **(D, G) -consistent path**.

D et G partagent les mêmes nœuds mais pas la même structure

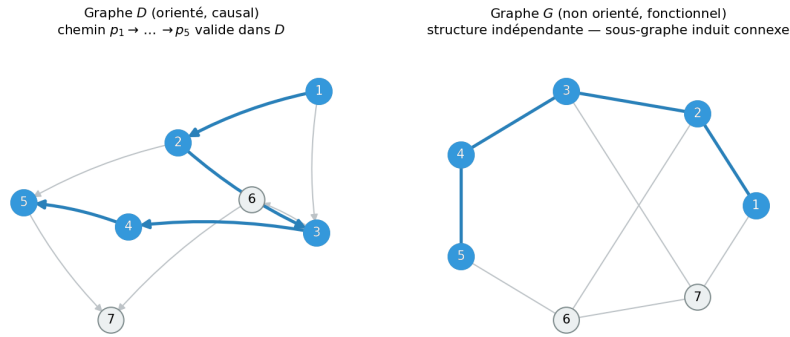


Fig. 1. Toy example on seven nodes. D and G share the same node set but represent different relationships. A valid solution must be a directed path in D while its vertices induce a connected subgraph in G .

2.1 Connectivity Cannot Be Checked Incrementally

The connectivity requirement is global and cannot in general be reduced to a local test between consecutive nodes. In particular, a valid solution may contain vertices that are not adjacent in G to their immediate predecessor but are connected to the selected set through other vertices.

An empirical audit of ILP2 optimal solutions also shows that intermediate prefixes do not generally provide a reliable basis for enforcing the final connectivity condition. Therefore, simply rejecting every candidate that temporarily becomes disconnected in G can eliminate valid solutions.

Our decoder consequently uses G only as a soft guidance signal during generation. The final connectivity constraint is checked exactly after a candidate directed path has been generated.

3 Methodology: One-to-one Skew-GRAM

Our approach contains two main components: an embedding model trained on directed random walks over D , and a guided decoder that combines the learned representations with structural information from D and G .

The complete procedure contains four phases:

1. random-walk generation and hub sub-sampling;
2. Skip-Gram-style embedding learning;
3. DAG-aware guided decoding;
4. exact extraction of the (D, G) -consistent sub-path.

3.1 General Approach: from Instance to Path, via Encoder and Decoder

The central analogy is:

node $\leftrightarrow$ word, random walk $\leftrightarrow$ sentence, context $\leftrightarrow$ graph neighborhood in a walk.

Random walks are generated only on D , following outgoing arcs. This ensures that the learned corpus reflects the directional structure required by the final path.

A sliding window is then used to generate center-context pairs. High-frequency nodes are sub-sampled to reduce the influence of hubs.

The resulting pairs are used to train two embedding matrices using negative sampling. Unlike standard Skip-Gram, negative examples are selected structurally from non-successors in D .

During decoding, the learned embeddings are combined with an exact dynamic-programming lookahead and a soft G -connectivity bonus. The decoder is executed repeatedly from different starting nodes, and the best candidate is retained.

Finally, because the guidance signals do not guarantee global connectivity, an exact sliding-window Union-Find procedure extracts the longest contiguous sub-path that is connected in G .

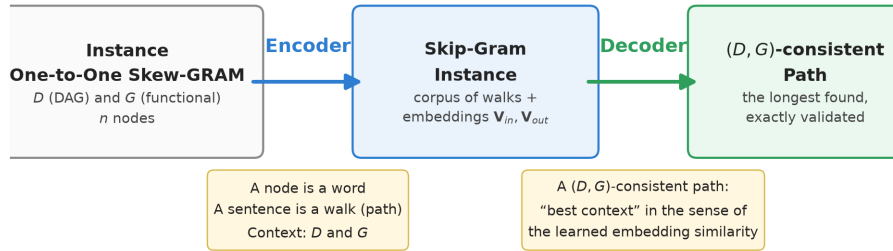


Fig. 2. General approach of One-to-one Skew-GRAM. The encoder transforms the instance into a Skip-Gram-like learning problem, while the decoder uses the learned embeddings and graph-specific information to generate and validate candidate paths.

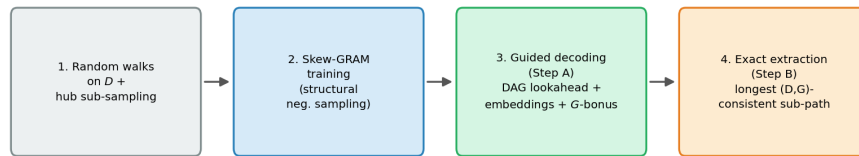


Fig. 3. Four-phase One-to-one Skew-GRAM pipeline, executed once per instance.

3.2 Phase 1: Random Walks and Hub Sub-sampling on D

Random walks are generated by following only outgoing arcs of D . For a current node u , the next node is sampled from its successor set $N^+(u)$. The walk terminates when a sink is reached or when the maximum walk length is attained.

The resulting walks are interpreted as sentences. A context window is then applied to generate center-context pairs.

High-frequency nodes are down-weighted using the standard word2vec sub-sampling strategy. This reduces the dominance of hubs during representation learning without changing the graph used by the decoder.

3.3 Phase 2: One-to-one Skew-GRAM Embedding Model

Two embedding matrices

$$\mathbf{V}_{in}, \mathbf{V}_{out} \in \mathbb{R}^{N \times d}$$

are trained, where $N = |V|$ and $d = 32$.

For a center node c , a positive context node o , and K negative samples $n_1, \dots, n_K$, the negative-sampling loss is

$$\mathcal{L}(c, o, n_{1:K}) = -\log \sigma(\mathbf{v}_{in,c}^\top \mathbf{v}_{out,o}) - \sum_{k=1}^K \log \sigma(-\mathbf{v}_{in,c}^\top \mathbf{v}_{out,n_k}). \quad (3)$$

The main modification to the standard Skip-Gram objective is **structural negative sampling**. Negative nodes are sampled among nodes that are not successors of the center node in D .

This makes the negative signal sensitive to the directional structure of the problem rather than relying only on frequency.

The similarity used during decoding is

$$\text{sim}(u, v) = \mathbf{v}_{in,u}^\top \mathbf{v}_{out,v}. \quad (4)$$

The model is trained independently for each instance because node identifiers do not have a common semantic meaning across the independently generated graphs.

3.4 Phase 3: DAG-Aware Guided Decoding (Step A)

A decoder based only on embedding similarity can select locally attractive successors that quickly lead to sinks. To reduce this behavior, we compute an exact longest-path lookahead on D .

For each node v , define

$$\text{lf}(v) = \begin{cases} 0, & N^+(v) = \emptyset, \\ \max_{w \in N^+(v)} (1 + \text{lf}(w)), & \text{otherwise.} \end{cases} \quad (5)$$

Since D is a DAG, all $\text{lf}(v)$ values are computed in a single reverse topological pass in

$$O(|V| + |A|).$$

At each decoding step, let u be the current node and $v \in N^+(u)$ a candidate successor. We define

$$S(u, v) = \frac{\text{lf}(v)}{\tau_{lf}} + \beta \frac{\text{sim}(u, v)}{\tau} + \log(\gamma) \mathbb{I}[v \text{ is adjacent in } G \text{ to a selected node}]. \quad (6)$$

The next node is sampled according to

$$P(v \mid u) = \frac{\exp(S(u, v))}{\sum_{w \in N^+(u)} \exp(S(u, w))}. \quad (7)$$

The three terms respectively represent:

1. the remaining directed-path potential in D ;
2. the learned embedding compatibility;
3. a soft preference for candidates connected in G to the path constructed so far.

Already visited nodes are masked to preserve the one-to-one constraint. The decoder is repeated 30,000 times per instance using different starting points, and the best final extracted path is retained.

3.5 Phase 4: Exact Extraction of the (D, G) -Consistent Sub-path (Step B)

The candidate generated in Phase 3 is guaranteed to be a directed path in D , but it is not necessarily connected in G .

For a candidate path

$$P = (p_1, \dots, p_L),$$

we therefore search for the longest contiguous interval $[i, j]$ such that

$$G[\{p_i, \dots, p_j\}]$$

is connected.

A contiguous sub-path of a valid D -path remains a valid D -path. Thus, this final step only needs to enforce the G -connectivity condition.

We use a sliding-window procedure combined with an incremental Union-Find structure [9]. For a candidate path of length L , the worst-case complexity is

$$O(L^2 \alpha(L)),$$

where α is the inverse Ackermann function.

This final step is exact: every path returned by the algorithm satisfies the definition of a (D, G) -consistent path.

4 Results and Discussion

4.1 Experimental Setup

We evaluate the proposed method on 38 synthetic Erdős–Rényi instances [10], each containing 100 nodes. An ILP2 optimum is available for 36 of the 38 instances.

The comparison includes:

- GNN DL2 [2];
- ILP2 [1];
- One-to-one Skew-GRAM.

The embedding dimension is 32, with 40 walks per starting node, a maximum walk length of 30, a context window of 3, and five structural negative samples per positive pair. The decoder performs 30,000 Monte-Carlo walks per instance.

All experiments use the fixed random seed **SEED=42**. The complete hyperparameter configuration is provided in Appendix A.

Table 1. Comparison of methods on 38 synthetic instances with 100 nodes. ILP2 optima are available for 36 instances.

Method	Mean Length	Mean Time	Speedup
GNN DL2 [2]	≈ 1.2	N/A	–
ILP2 [1]	9.97	28.0 s	reference
One-to-one Skew-GRAM	7.13	2.7 s	$\sim 10\times$

The proposed method obtains an average path length of 7.13 nodes, compared with 9.97 nodes for ILP2. The corresponding average relative difference is approximately

$$\frac{7.13 - 9.97}{9.97} \approx -28.5\%.$$

The mean runtime decreases from 28.0 seconds for ILP2 to 2.7 seconds for One-to-one Skew-GRAM, corresponding to a speedup of approximately $10.4\times$.

The results therefore indicate a substantial quality–time trade-off: the proposed heuristic does not consistently recover the exact optimum, but it obtains considerably longer paths than the previous GNN approach while requiring substantially less computation than ILP2.

Table 2 illustrates the variability of the heuristic across instances. One-to-one Skew-GRAM exactly matches ILP2 on several instances, including 100_121,

Table 2. Per-instance comparison between One-to-one Skew-GRAM and ILP2 on ten representative 100-node instances.

Instance	ILP2 Skew-GRAM (nodes)	(nodes)	Gap ILP2 time (%)	Skew-GRAM time (s)	time (s)
100_121	6	6	0.0%	19.59	0.69
100_402	11	11	0.0%	13.16	0.80
100_190	11	11	0.0%	25.44	0.77
100_197	11	9	-18.2%	19.73	0.70
100_14	12	9	-25.0%	8.51	0.77
100_320	12	8	-33.3%	33.91	0.80
100_128	13	8	-38.5%	18.44	0.90
100_66	16	9	-43.8%	9.98	0.97
100_388	15	8	-46.7%	58.18	0.84
100_527	11	4	-63.6%	15.83	1.04

100_402, and 100_190. Across the 36 instances with a known optimum, it matches the ILP2 optimum on 13 instances.

The runtime remains substantially lower for all instances shown. The difference is particularly visible for instances such as 100_388, where ILP2 requires 58.18 seconds while the proposed method requires only 0.84 seconds.

The variability in solution quality is expected because the decoder relies on stochastic sampling. Some instances contain promising paths that are easier to reach through the learned guidance, while others require exploration of less probable branches.

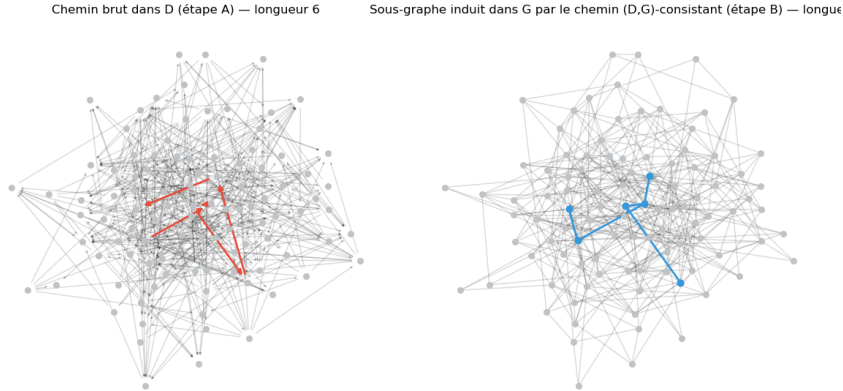


Fig. 4. Example of the two-step decoding process. Step A generates a directed path in D , while Step B extracts the longest contiguous sub-path that is connected in G .

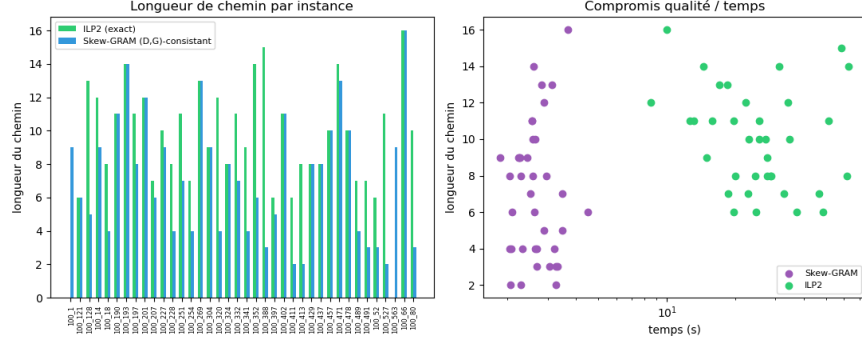


Fig. 5. Per-instance path-length comparison between One-to-one Skew-GRAM and ILP2, together with the quality–time trade-off across the evaluated instances.

4.2 Ablation of DAG Lookahead

We also evaluate the importance of the exact DAG lookahead by removing it from the decoder.

Without the lookahead, decoding based primarily on embedding similarity produces an average path length of 5.87 nodes, compared with 7.13 nodes for the complete approach.

This represents a reduction of approximately 17.7%.

The result confirms that embedding similarity alone does not adequately capture the remaining path potential in the DAG. A candidate node may be locally compatible with the current node while leading to a short branch or a sink.

The dynamic-programming lookahead provides this information explicitly and therefore helps the decoder avoid premature dead ends.

4.3 Discussion

The results highlight the benefit of combining learned and algorithmic components.

First, the large difference between DL2 and One-to-one Skew-GRAM suggests that purely sequential neural decoding has difficulty maintaining sufficiently long paths under the coupled constraints of D and G .

Second, the DAG lookahead provides exact information that can be computed efficiently because D is acyclic. Rather than requiring the embedding model to learn this property implicitly, we provide it directly to the decoder.

Third, G is deliberately used as a guidance and validation structure rather than being incorporated into the embedding objective. This keeps the representation-learning problem focused on the directed structure that defines valid path order, while the global connectivity constraint is handled explicitly during decoding and extraction.

The proposed method should therefore not be interpreted as a replacement for ILP2 when guaranteed optimality is required. Its main advantage is the quality–time trade-off: it obtains substantially longer paths than the previous GNN approach while being approximately ten times faster than the exact solver on the evaluated instances.

5 Conclusion

We presented **One-to-one SkewGRAM**, a hybrid approach for the NP-hard One-to-one SkewGRAM problem. The method combines Skip-Gram-style node embeddings learned from directed random walks over D with an exact DAG lookahead, a connectivity-guided decoding strategy, and an exact final extraction step based on Union-Find.

The experimental results on 38 synthetic 100-node instances show that the proposed approach provides a strong quality–time trade-off. It obtains an average path length of 7.13 nodes in 2.7 seconds, compared with 9.97 nodes in 28.0 seconds for the exact ILP2 solver. It reaches the ILP2 optimum on 13 of the 36 instances for which an optimum is available, while requiring substantially less computation. The ablation study also confirms the importance of the DAG lookahead for maintaining longer candidate paths.

These results suggest that combining learned node representations with problem-specific graph algorithms can provide an effective alternative to both purely exact optimization and end-to-end neural decoding for coupled-graph combinatorial problems.

A Hyperparameters

Table 3 gives the hyperparameters used to obtain the reported results.

Table 3. Hyperparameters used by the encoder and decoder.

Parameter	Symbol	Value	Role
<i>Encoder – Phases 1–2</i>			
Walks per starting node	–	40	Size of the walk corpus over D
Maximum walk length	–	30	Early stop on a sink
Context window	–	3	Center-context pair generation
Embedding dimension	d	32	Size of node embeddings
Training epochs	–	8	Adam optimizer
Structural negatives per pair	K	5	Negative samples
Learning rate	–	0.01	Adam learning rate
<i>Decoder – Phases 3–4</i>			
Monte-Carlo decoding walks	–	30,000	Number of candidate paths
Embedding-signal temperature	τ	0.5	Embedding score temperature
Embedding-signal weight	β	0.3	Weight of embedding similarity
G -connectivity bonus	γ	6.0	Connectivity preference
DAG-lookahead temperature	π_f	1.5	Lookahead score temperature

References

1. Ahmed Sidi, M. L., Babou, H. M., Dhib, C., Neron, E., Soukhal, A., Nanne, M. F.: Constraint programming approaches for finding conserved metabolic and genomic patterns. *Computers & Operations Research*, 183:107166 (2025)
2. Sidi, M. L. A., Babou, H. M., Dhib, C.: Deep Learning approaches for finding a longest DG-consistent path in biological networks. Working paper / Preprint (2024)
3. Zaharia, A., et al.: CoMetGeNe: Mining conserved neighborhood patterns in metabolic and genomic contexts. *BMC Bioinformatics*, 20(1):1–20 (2019)
4. Mikolov, T., Sutskever, I., Chen, K., Corrado, R., Dean, J.: Distributed Representations of Words and Phrases and their Compositionality. In: *Advances in Neural Information Processing Systems* (2013)
5. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: Online Learning of Social Representations. In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2014)
6. Grover, A., Leskovec, J.: node2vec: Scalable Feature Learning for Networks. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (2016)
7. Sharan, R., Ideker, T.: Modeling protein interaction networks. *Nature Reviews Genetics*, 7(7):575–582 (2006)
8. Fertin, G., Babou, H. M., Rusu, I.: Algorithms for sub-network mining in heterogeneous networks. *Lecture Notes in Computer Science*, 7276:184–194 (2012)
9. Tarjan, R. E.: Efficiency of a Good But Not Linear Set Union Algorithm. *Journal of the ACM*, 22(2):215–225 (1975)
10. Erdős, P., Rényi, A.: On Random Graphs I. *Publicationes Mathematicae*, 6:290–297 (1959)
11. Bellman, R.: On a Routing Problem. *Quarterly of Applied Mathematics*, 16(1):87–90 (1958)
12. Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. *Advances in Neural Information Processing Systems*, 28 (2015)
13. Bello, I., Pham, H., Le, Q. V., Norouzi, M., Bengio, S.: Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940* (2016)
14. Dai, H., Khalil, E. B., Zhang, Y., Dilkina, B., Song, L.: Learning combinatorial optimization algorithms over graphs. *Advances in Neural Information Processing Systems*, 30 (2017)
15. Karalias, K., Loukas, A.: Erdős goes neural: an unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33:3135–3145 (2020)
16. Kingma, D. P., Ba, J.: Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014)